Lab 0: Toolchain Setup and First Flash

Objectives

- To install an ARM development toolchain, and to make sure that it can build code for the target microcontroller.
- To identify the on-board debug probe (ST-LINK) and the target microcontroller (STM32G491RE). These are two different chips on the same board.
- To create, build, and flash a minimal project, and to see the program run on the board.
- To learn the memory map: flash, RAM, and peripherals have different, fixed address ranges.
- To measure a signal with an oscilloscope.

Required Hardware and Software

- A NUCLEO-G491RE development board and a USB cable. The USB connector supplies power, and it is also the ST-LINK debug interface.
- An oscilloscope (one channel minimum), for Step 8.
- A computer with an internet connection, to download the toolchain.
- Visual Studio Code, with the STMicroelectronics extension **STM32CubeIDE for Visual Studio Code**. The extension includes its own ARM GCC toolchain, CMake, Ninja, and STM32CubeProgrammer. It downloads them when you accept its prerequisites prompt in Step 4.
- **STM32CubeMX**, which you install separately from the ST website (link in Step 1).

Each of the five tools has one function:

- **STM32CubeMX** reads the pin and clock configuration that you select in the board selector of Step 2. Then it generates the project: the `Core/` sources, the `Drivers/` tree, and the build files, with `CMakeLists.txt`.
- **ARM GCC** is the cross-compiler. It changes your C source into machine code for the ARM Cortex-M4 core of the STM32G491RE. It is the GNU Compiler Collection of most desktop Linux systems, configured for ARM processors.
- **CMake** reads the `CMakeLists.txt` of the project (Figure 19, in Step 2), which CubeMX writes. From this file, CMake generates the low-level build instructions.
- **Ninja** executes the build instructions from CMake. It starts ARM GCC on each source file, in the correct sequence, and only for the files that changed after the last build.
- **STM32CubeProgrammer** communicates with the ST-LINK over USB, and writes your compiled binary into the flash memory of the target. This is the “flash” step, the first step that accesses the board.

A naming collision to watch for

STMicroelectronics uses the name STM32CubeIDE for two different products. The first product is a standalone, Eclipse-based application. The second product is the VS Code extension of this course, **STM32CubeIDE for Visual Studio Code**. This handout calls the first product “classic STM32CubeIDE”, and the second product “the extension”.

Course and Hardware Overview

What this course is

This course teaches embedded systems programming on the STM32G491RE microcontroller, on the NUCLEO-G491RE development board. From Lab 1 onward, each lab configures a peripheral first through direct register access. Then it configures the peripheral through the hardware abstraction layer (HAL) of STMicroelectronics. You get the address, the bit layout, and the reset value of each register from the reference manual, RM0440.

Two chips, one board

The NUCLEO-G491RE board has two different microcontrollers:

- **ST-LINK/V3E**: a debug probe. It connects to your computer through USB. It changes the USB signals into SWD, the low-level debug protocol of the target chip.
- **STM32G491RE**: the target, the chip that each lab programs. It has its own flash memory, RAM, and peripherals.

If a problem occurs, first find which of the two chips has the problem. A driver problem or a connection problem is in the ST-LINK. A logic error is in the code that runs on the target.

SWD (Serial Wire Debug) is the two-wire debug interface of ARM, in the Cortex-M4 core. One wire carries a clock (`SWCLK`). The other wire carries data in the two directions (`SWDIO`).

Through these two wires, the ST-LINK can halt the processor, and read and write each memory address or register. It can also stop at breakpoints and write a new program into flash. For these operations, the ST-LINK does not need the help of the program that runs on the target. Each flash, breakpoint, and register view in Steps 5 to 7 uses SWD. The two wires connect the ST-LINK and the STM32G491RE on the board.

STM32G491RE specifications

Item	Value
Core	ARM Cortex-M4F with hardware floating-point unit
Clock at reset	16 MHz internal oscillator (HSI), maximum 170 MHz
Flash	512 KB, starts at `0x0800 0000`
SRAM	112 KB, starts at `0x2000 0000`
Peripheral region	`0x4000 0000` to `0x5FFF FFFF`
User LED (LD2)	PA5, active high
User button (B1)	PC13. Lab 1 gives the polarity for this board.
Debugger	Integrated ST-LINK/V3E

The core, clock, and memory values in this table come from the reference manual of the STM32G491RE (RM0440) and from its datasheet. The LED, button, and debugger rows come from the user manual of the board (UM2505). You can also open the two manuals from the Docs & Resources tab of STM32CubeMX (Figure 14). You open UM2505 there in Step 2.

Other reference documents

In addition to RM0440, three references describe the three levels of the code in this course, from the lowest level to the highest level:

- **PM0214**, the STM32 Cortex-M4 programming manual, describes the instruction set of the Cortex-M4. These instructions are the assembly operations that the processor executes, for example `NOP` (no operation). The *Instruction set summary* chapter lists and describes each instruction, in alphabetical sequence.
- The **CMSIS-Core** documentation describes the C-level core access functions and intrinsics, between assembly and the HAL. For example, `__NOP()` puts the same no-operation instruction into C code. The intrinsics are under *API Reference* → *Intrinsic Functions for CPU Instructions*.
- **UM2570**, the description of the STM32G4 HAL and low-layer drivers, describes each `HAL_*` and `LL_*` function. An example is `HAL_GPIO_TogglePin()`, which Step 3 uses. Each driver has its own chapter (for example, *GPIO Generic Driver*), and each function has its own subsection.

The memory map

Each address that a program accesses is in one of a small number of fixed regions. This is true for an instruction fetch, a stack push, and a register write. Figure 1 shows the three regions that this course uses most.

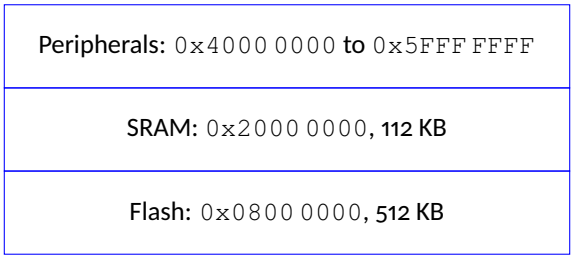


Figure 1: the three address regions that this course uses most. Flash holds your compiled program. SRAM holds its variables, heap, and stack. The peripherals are at memory addresses. Higher addresses are higher in the figure, here and in Figure 2.

The linker script that CubeMX generates divides SRAM into statically allocated globals (`.data` and `.bss`), a heap, and a stack (Figure 2). A different linker script can divide SRAM differently.

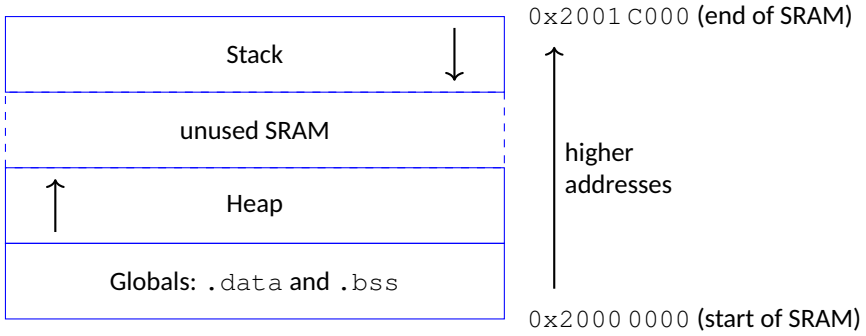


Figure 2: the layout of SRAM, from the linker script. The stack starts at the top of SRAM and grows down with each function call. The heap starts immediately above the statically allocated globals and grows up with each allocation. If the two meet, memory corruption occurs with no error message.

The heap is reserved space. It stays unused until the code calls `malloc()`, or a library function that calls `malloc()` (some `printf()` implementations do this). The code of Lab 0 does not call `malloc()`, thus its heap stays unused.

Fact: each STM32 family has its own memory map

STM32 microcontrollers are in **families**, for example the **STM32F4** and the **STM32G4**. The STM32G491RE of this course is in the **G4** family. The families have the same peripherals, but **a peripheral can be at a different address in a different family**.
The AHB (Advanced High-performance Bus) is the on-chip bus that connects the processor core to its peripherals. There is more than one AHB, each with a number. The AHB of a peripheral sets a part of its address, and the bit in `RCC` that enables its clock. The GPIO ports show the difference:

- **STM32F4**: the GPIO clocks are on `AHB1`, and `GPIOA` starts at `0x4002 0000`.
- **STM32G4**: the GPIO clocks are on `AHB2`, and `GPIOA` starts at `0x4800 0000`.

Most STM32 example code on the internet is for the F4. Take each address from the documents of the **G4** family, for example RM0440.

Course Materials on GitHub

The course has a public GitHub repository. It contains each lab handout:

`https://github.com/YuvalHarary/Embedded_Lab_STM32`

Star the repository from your GitHub account (the star icon in Figure 3). The repository then goes into the starred list of your account, and you can open it from there.

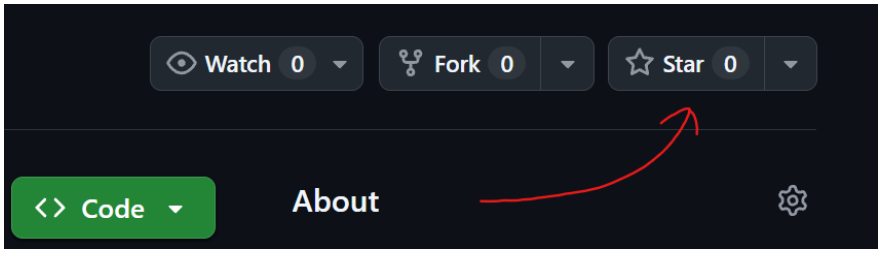


Figure 3: the star icon of the repository.

Figure 4 shows the layout of the repository.

```
course-repo/
├── README.md
├── lab00-setup/
│   └── lab00.pdf
├── lab01-gpio/
│   └── lab01.pdf
└── ...
```

Figure 4: the course GitHub repository, as the student sees it.

Report a Problem with the Course Materials

If a handout or other content is wrong or missing, open an issue on GitHub. Before you open an issue, look at the open issues for the same problem.

1. Open the **Issues** tab of the repository.

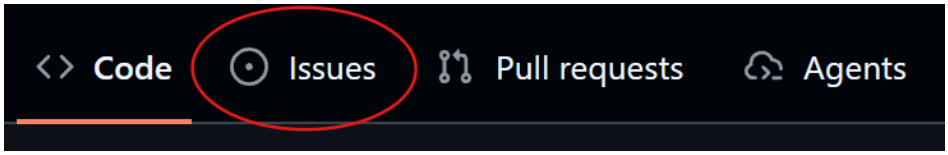


Figure 5: the Issues tab, next to Code and Pull requests.

2. Click **New issue**.

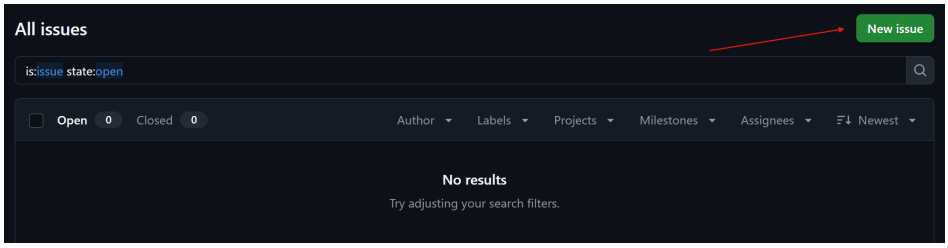


Figure 6: the New issue button in the issues list.

3. Give the issue a clear title. In the description, explain the problem, and give the file, the page, or the line. Then click **Create**.

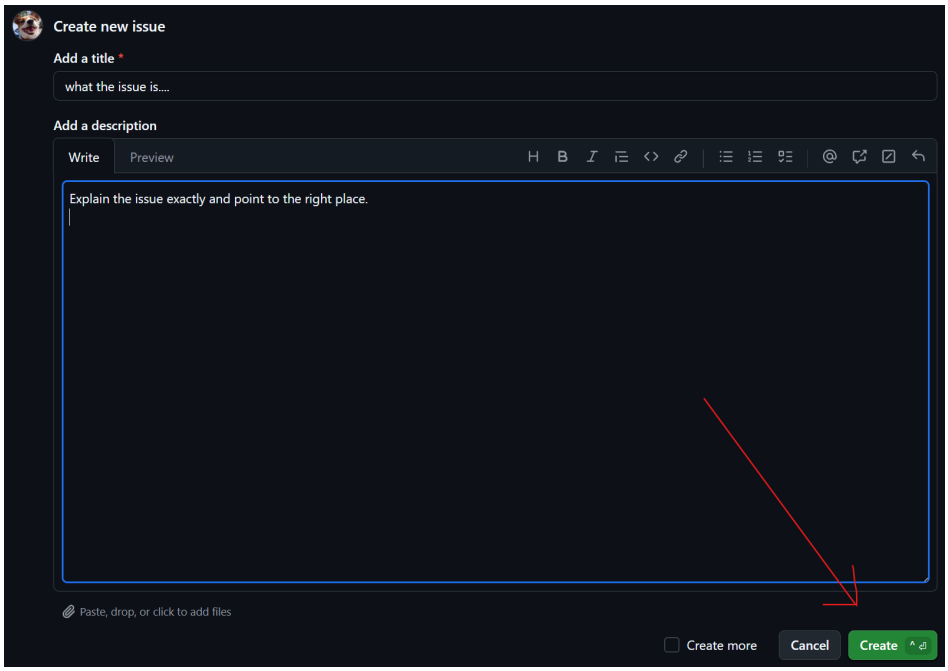


Figure 7: a new issue, with its title and description.

Step 1: Install Visual Studio Code, the Extension, and STM32CubeMX

1. Install Visual Studio Code for your operating system from its official distribution, if it is not installed.
2. Open the Extensions view. Install **STM32CubeIDE for Visual Studio Code** (from STMicroelectronics).

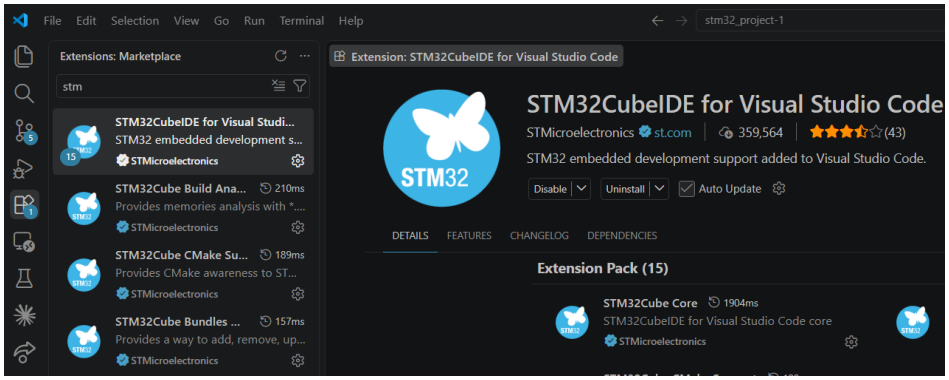


Figure 8: the extension, installed, in the Extensions view.

3. Install the classic STM32CubeMX from the ST website. It is a different application from the VS Code extension. Its preview successor, MX2, does not support the STM32G4 family.

   https://www.st.com/en/development-tools/stm32cubemx.html

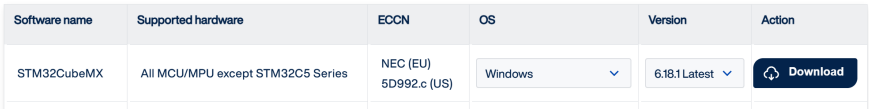


Figure 9: the STM32CubeMX row of the development-tools download page of ST.

4. Link the extension to STM32CubeMX. You install the two applications separately. Thus, you must give the location of STM32CubeMX to the extension, one time for each computer.

   (a) In VS Code, open the STM32Cube extension panel in the activity bar. Click **Launch STM32CubeMX**. If STM32CubeMX opens, the link is correct.

   (b) If the notification in Figure 10 appears, click **Find STM32CubeMX**.

   (c) In the file picker, open the STM32CubeMX installation folder. Select `STM32CubeMX.exe`. Click **Open** (Figure 11). STM32CubeMX then opens.

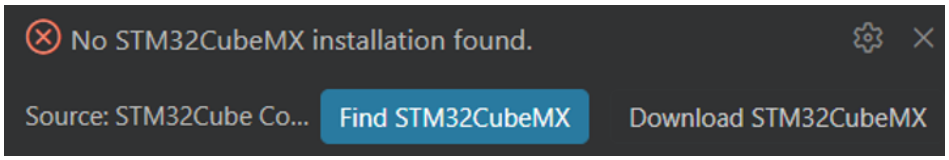


Figure 10: the extension cannot find STM32CubeMX.

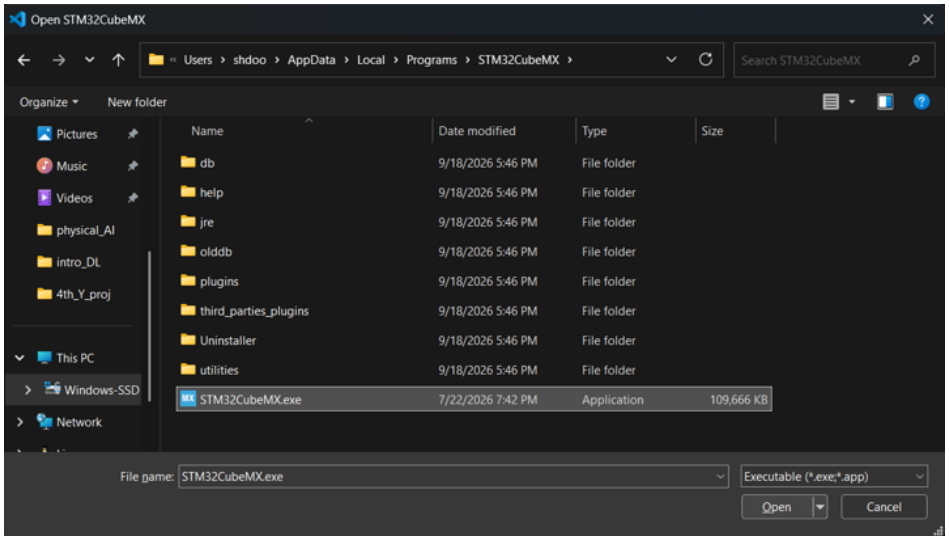


Figure 11: the selection of `STM32CubeMX.exe`. For an installation for one user, the folder is `%LOCALAPPDATA%\Programs\STM32CubeMX`.

5. Connect the NUCLEO-G491RE board to the computer with a USB cable. Windows usually installs the ST-LINK drivers automatically.

6. Make sure that VS Code detects the board. Open the **Run and Debug** panel in the activity bar. Look at the **STM32CUBE DEVICES AND BOARDS** section. If it shows *No device detected* (Figure 31), install the drivers manually, as Troubleshooting shows.

Step 2: Create a New STM32 Project

1. Start STM32CubeMX. On Windows, type `STM32CubeMX` in the search bar. Open the application.

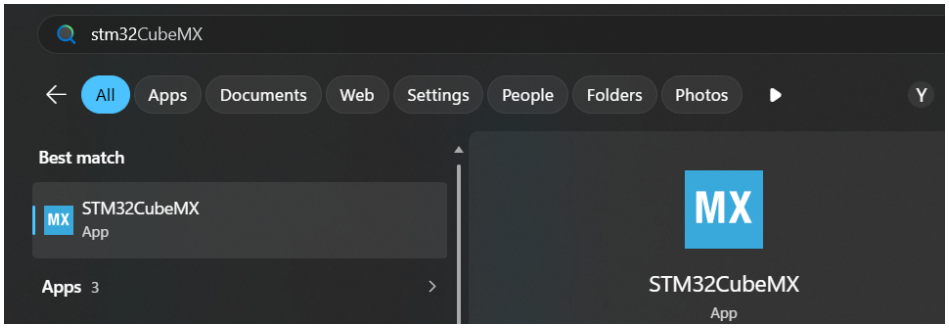


Figure 12: STM32CubeMX in the Windows search bar.

2. In STM32CubeMX, select **Start My Project from ST Board**. Type the board name, **NUCLEO-G491RE**, in the **Commercial Part Number** search box.

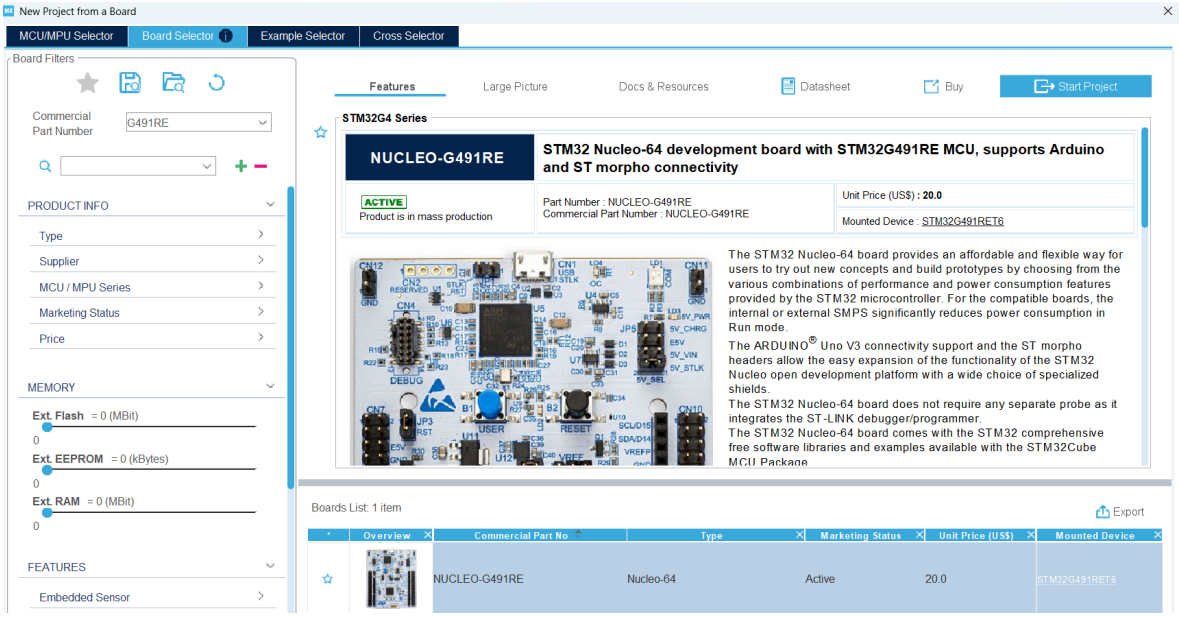


Figure 13: the Board Selector, with the filter G491RE, before the start of a project from NUCLEO-G491RE.

3. Before you start the project, open the user manual of the board from the Board Selector. Select the **Docs & Resources** tab. Expand **User Manuals**. Open **UM2505** (STM32G4 Nucleo-64 boards). Save a copy: it shows what is connected to what on this board.

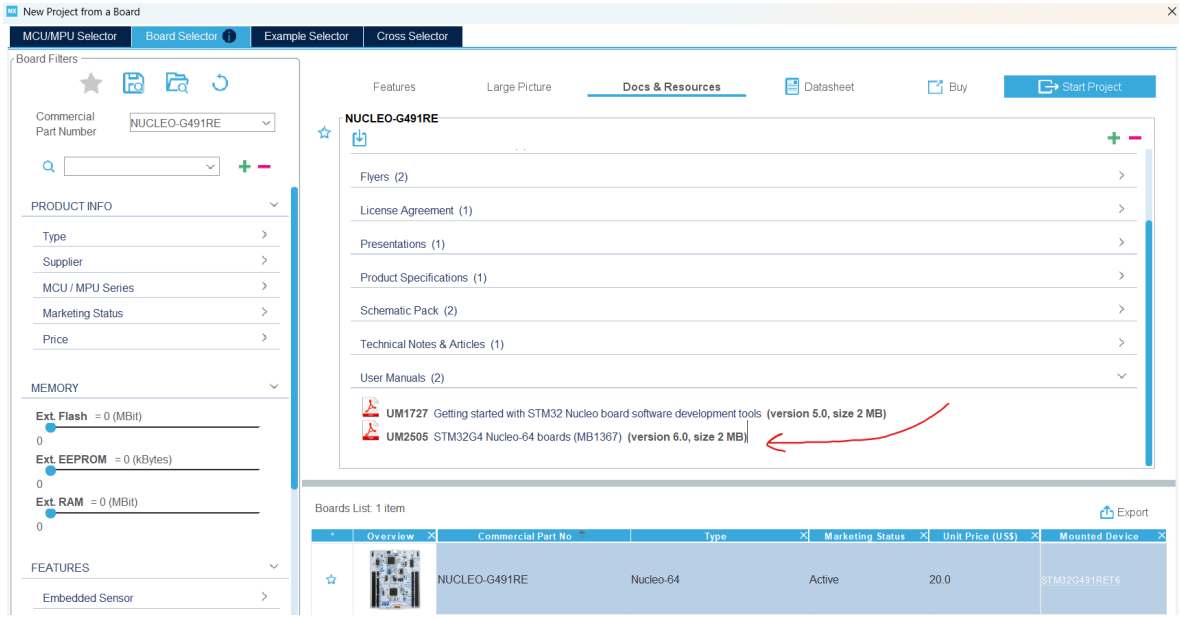


Figure 14: the Docs & Resources tab of the Board Selector, with UM2505 under User Manuals.

For example, in the description of LD2, the manual gives the microcontroller pin that drives the LED:

LD2 USER

This green LED is a user LED connected to STM32G4 I/O PA5 (SB6 ON) corresponding to the ARDUINO® D13. To light LED LD2, a high logic state "1" must be written in the corresponding GPIO PA5. A transistor is used to drive the LED LD2. LD2 consumption does not impact the VDD STM32G4 power measurement, since LD2 is isolated from it.

Figure 15: the description of LD2 in UM2505: PA5, on Arduino D13.

The board diagrams of the manual label each connector, jumper, and component with the name that is printed on the board:

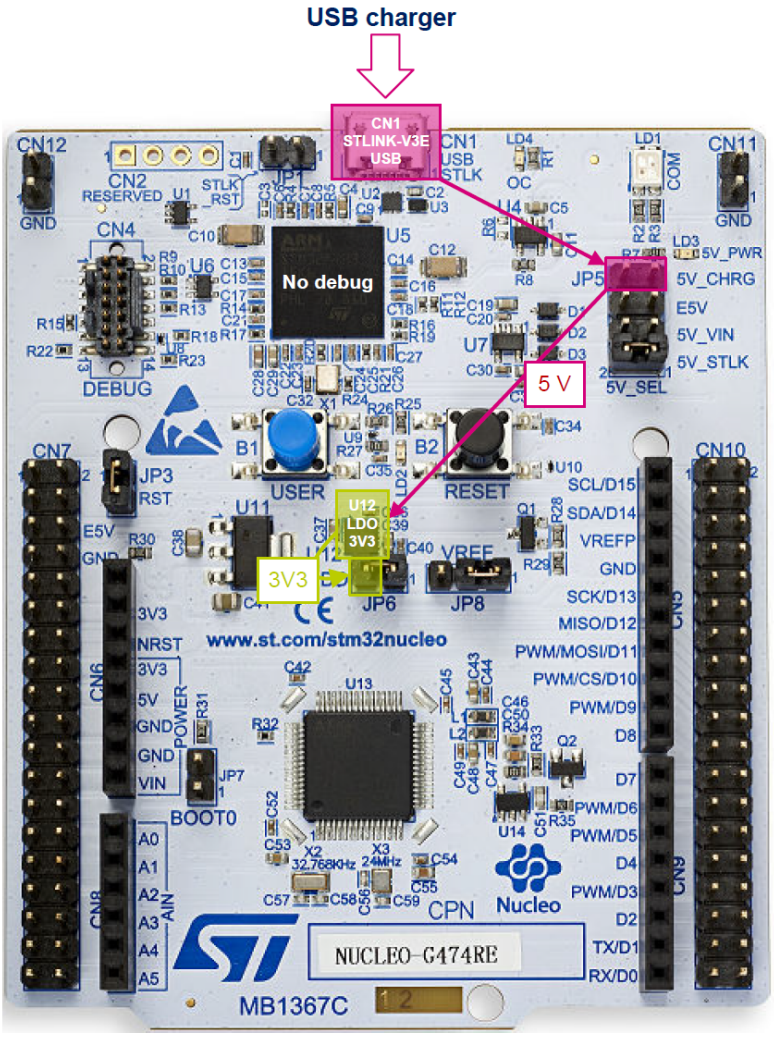


Figure 16: a board diagram from UM2505. The manual is for all the STM32G4 Nucleo-64 boards, which have the same board layout (MB1367C). Thus, some figures show a different board, for example the NUCLEO-G474RE.

In the Board Selector, click **Start Project**.

4. After **Start Project**, the **Board Project Options** dialog opens (Figure 17). Its **Select software components** section lists the BSP (Board Support Package) drivers:

- A BSP driver is code for one board. An example is a `BSP_LED_On()` function for the LED of this board.
- A BSP driver uses the HAL, the hardware abstraction layer of ST. Lab 1 explains the HAL.

Set the dialog as follows:

- Make sure that **Generate demonstration code** is not selected.
- Make sure that the BSP driver checkboxes **USER LED GREEN (LD2)**, **USER B1**, and **Virtual Com Port** are not selected.
- Click **OK**.

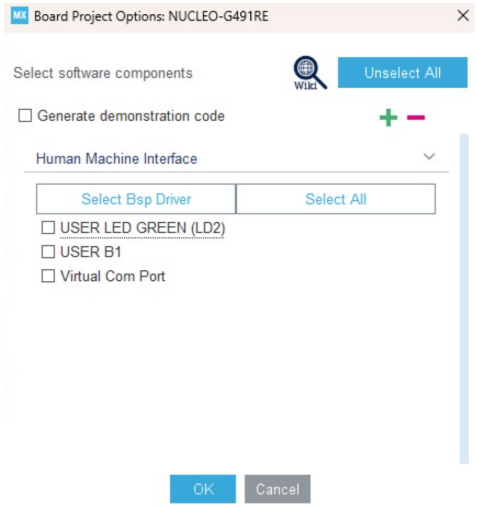


Figure 17: the Board Project Options dialog. Demonstration code and all BSP drivers are not selected.

5. Select the **Project Manager** tab. Type a **Project Name** and a **Project Location**. Make sure that **Toolchain / IDE** is set to **CMake**. A **Default Compiler/Linker** field then appears. Set it to **GCC**. Then click **GENERATE CODE**.

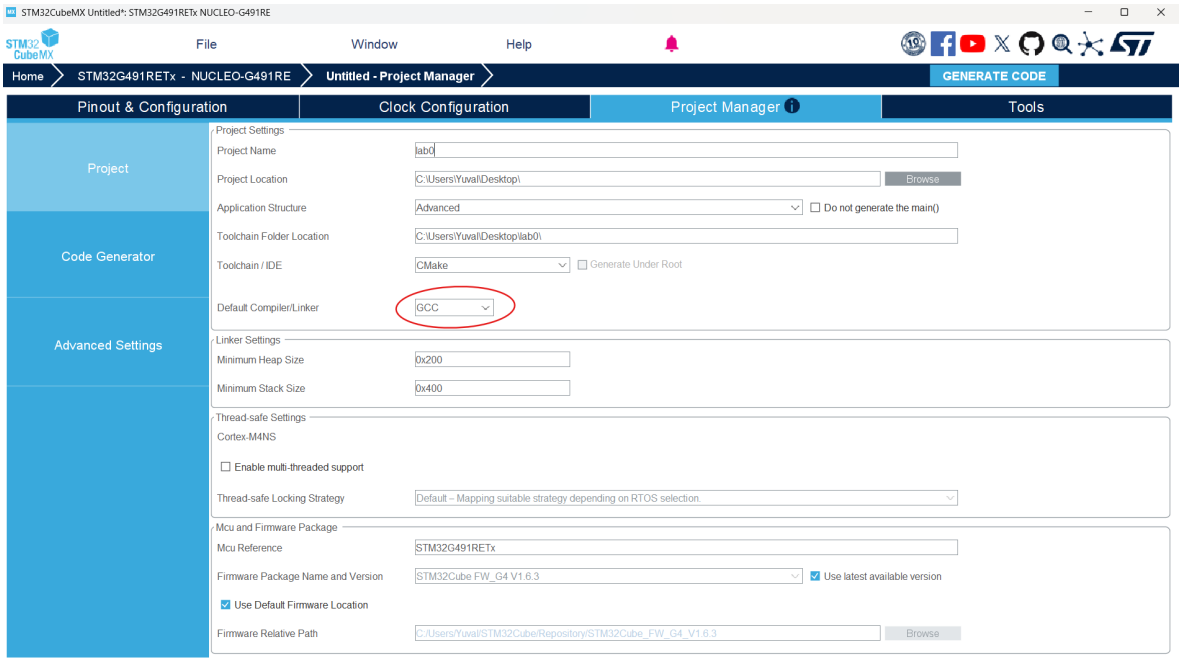


Figure 18: the Project Manager tab, with the name and the location of the project, Toolchain / IDE set to CMake, and Default Compiler/Linker set to GCC.

6. Open the folder of the generated project in VS Code (**File** → **Open Folder**). The extension automatically recognizes the CMake project. When you open the folder, the extension can also show the prerequisites prompt of Step 4. Accept it.

7. The generated project has the layout of Figure 19.

```
your-project-name/
├── Core/
│   ├── Inc/
│   │   ├── main.h
│   │   ├── stm32g4xx_hal_conf.h
│   │   └── stm32g4xx_it.h
│   └── Src/
│       ├── main.c
│       ├── stm32g4xx_hal_msp.c
│       ├── stm32g4xx_it.c
│       ├── syscalls.c
│       ├── sysmem.c
│       └── system_stm32g4xx.c
├── Drivers/
│   ├── CMSIS/
│   └── STM32G4xx_HAL_Driver/
├── cmake/
│   └── stm32cubemx/CMakeLists.txt
├── CMakeLists.txt
├── CMakePresets.json
├── startup_stm32g491xx.s
├── STM32G491XX_FLASH.ld
└── your-project-name.ioc
```

Figure 19: the file tree of the generated project.

`Core/Inc/` and `Core/Src/`: the code of your project.

- **`main.h`** Holds the declarations of the project, and the pin and port macros `LD2_Pin` and `LD2_GPIO_Port` that you add in Step 3.
- **`stm32g4xx_hal_conf.h`** Enables or disables each HAL peripheral module at compile time. Only the modules that your configuration uses are enabled.
- **`stm32g4xx_it.h`** Declares the interrupt handler functions of `stm32g4xx_it.c`.
- **`main.c`** Your application. The `USER CODE` markers show the only sections that CubeMX keeps when it generates the code again. You edit this file in Step 3.
- **`stm32g4xx_hal_msp.c`** The low-level pin and clock setup of the peripherals (MSP: MCU Support Package). Each `HAL_*_Init` function calls it automatically. You do not edit it in this lab.
- **`stm32g4xx_it.c`** The bodies of the interrupt handlers (SysTick, fault handlers, and other handlers). Most bodies are empty, because Lab 0 enables no interrupts of its own.
- **`syscalls.c`, `sysmem.c`** Minimal C library stubs (memory allocation, file I/O). The runtime library of the compiler links against them.

- **`system_stm32g4xx.c`** Supplies `SystemInit()`. The startup code calls `SystemInit()` before `main()` to prepare the core (floating-point unit access and the location of the vector table). `SystemClock_Config()` in `main.c` configures the clock tree later. ST supplies this file, as the CMSIS convention specifies.

`Drivers/`

- **`CMSIS/`** The core headers of Arm (register names, intrinsics) and the device header of ST for the STM32G491xx. This header defines each register name that the labs use from Lab 1 onward.
- **`STM32G4xx_HAL_Driver/`** The HAL source of STMicroelectronics, with one file pair for each peripheral family (GPIO, RCC, UART, and other families). The build includes only the files that the configuration of this project uses.

Build configuration.

- **`cmake/stm32cubemx/CMakeLists.txt`** Lists the source files above that belong to this project. CubeMX generates it again when the pin configuration changes.
- **`CMakeLists.txt`** The top-level build description. It connects your `Core/` sources and the generated driver sources.
- **`CMakePresets.json`** Defines the Debug and Release build configurations (compiler flags, build directory). The Build command of Step 4 uses them.
- **`startup_stm32g491xx.s`** The vector table and the reset handler in ARM assembly. This is the first code that runs when the chip gets power, before `main()`.
- **`STM32G491XX_FLASH.ld`** The linker script. It gives the linker the addresses of flash and RAM in the address space of the STM32G491RE (Figures 1 and 2). It also reserves the heap and the stack of these figures.
- **`*.ioc`** The CubeMX project file. It holds the configuration that you selected in Step 2, and CubeMX generates all the files above from it.

Step 3: Add the Minimal Blink

In this step, you write a program that makes the user LED LD2 blink. LD2 is connected to pin PA5 (Figure 15). You do three edits:

1. Give names to the pin of LD2.
2. Set PA5 as an output.
3. Toggle LD2 in a loop.

1. Give names to the pin of LD2. Open `Core/Inc/main.h`. Add these two macros between the `USER CODE BEGIN Private defines` and `USER CODE END Private defines` markers:

```
#define LD2_Pin GPIO_PIN_5
#define LD2_GPIO_Port GPIOA
```

Listing 1: Pin and port macros for LD2 (PA5), added by hand.

Where the hardware behind this listing is documented

The description of LD2 in UM2505 (Figure 15) states that PA5 drives the LED, active high. PA5 is also on the Arduino header pin with the label `SCK/D13`, which Step 8 probes.

2. Set PA5 as an output. In `Core/Src/main.c`, between the `USER CODE BEGIN 2` and `USER CODE END 2` markers, copy the lines below. They enable the clock of port *A* and set PA5 as a push-pull output with no pull resistor.

```
GPIO_InitTypeDef GPIO_InitStruct = {0};
__HAL_RCC_GPIOA_CLK_ENABLE();
GPIO_InitStruct.Pin = LD2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
```

Listing 2: Configure PA5 (LD2) as an output.

3. Toggle LD2 in a loop. In `Core/Src/main.c`, between the `USER CODE BEGIN WHILE` and `USER CODE END WHILE` markers, copy only the two lines below:

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(500);
```

Listing 3: Toggle LD2 and wait, forever.

Step 4: Build

Before you build the project, set it up and configure it through the extension. When you open the folder of the project, or when you use the extension for the first time, a notification appears. The notification offers to install the STM32Cube bundle prerequisites. If you accept it, the extension downloads the four tools that it manages (ARM GCC, CMake, Ninja, and STM32CubeProgrammer). Click **Yes**.

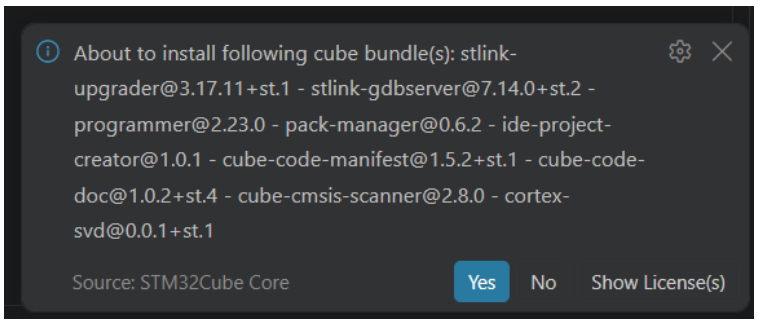


Figure 20: the notification for the STM32Cube bundle prerequisites.

If the notification does not appear, open the command palette (**Ctrl+Shift+P**). Run **STM32Cube: Install STM32Cube Bundle prerequisites**.

Open the STM32Cube panel in the VS Code activity bar. Click **Set Up STM32Cube project(s)**. If it is gray, see Troubleshooting.

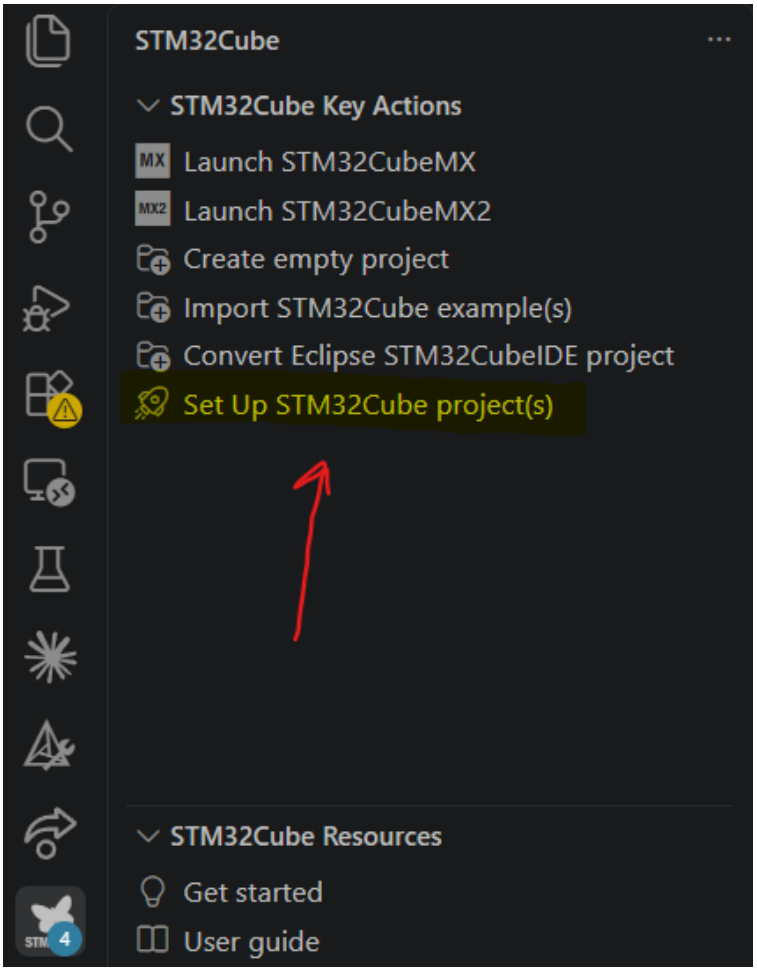


Figure 21: the Key Actions of the STM32Cube panel, with Set Up STM32Cube project(s) selected.

A **Project Setup** tab then opens. It shows that you must configure the CMake project before you build it. Select the **Debug** preset. Make sure that **Toolchain** is set to **GCC**, as in Step 2. Click **Configure**.

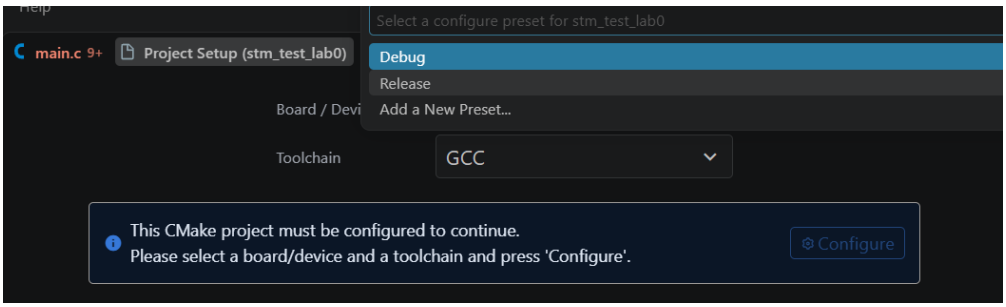


Figure 22: the Project Setup tab, with the Debug preset and the GCC toolchain selected, before a click on Configure.

The configuration fills in **Board / Device**, confirms **Toolchain**, and adds a Cortex-M4 target entry below. Click **Save and close**.

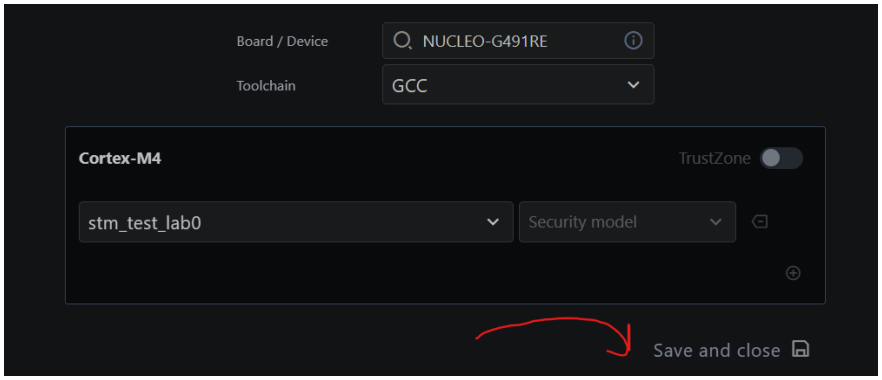


Figure 23: the Project Setup tab after the configuration, with Save and close.

Use the Build command of the extension, or the CMake build button in the VS Code status bar. The first build compiles the HAL and CMSIS sources and the lines from Step 3. Thus, it takes longer than each later build. A successful build shows a memory usage summary like this one:

```
Memory region         Used Size  Region Size  %age Used
             RAM:        1584 B       112 KB      1.38%
           FLASH:        5856 B       512 KB      1.12%
```

Listing 4: Expected output at the end of a successful build.

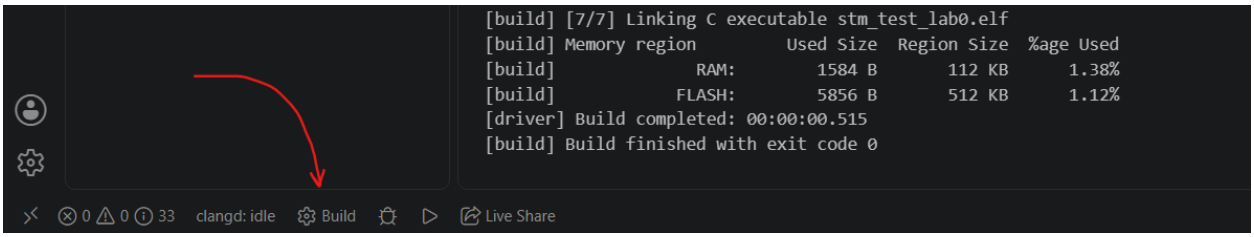


Figure 24: the Build button in the VS Code status bar, and its output at the end of the build.

RAM is the space in the 112 KB SRAM region that your global and static variables use, with the heap and stack reservations of Figure 2. **FLASH** is the space in the 512 KB flash region at `0x0800 0000` that your compiled code and constants use.

Step 5: Flash and Run

Before you use Run or Debug, open the **Run and Debug** panel in the VS Code activity bar. Click **create a launch.json file**. This generates the launch configuration that the Run and Debug commands of the extension need to connect to the ST-LINK.

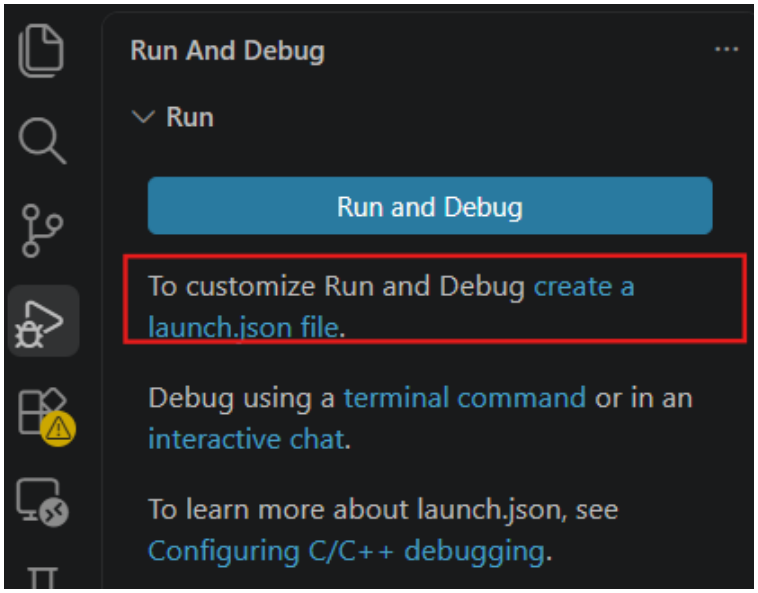


Figure 25: the Run and Debug panel, with create a launch.json file marked.

A **Select debugger** list then appears. Select **STM32Cube: STM32 Launch STLink GDB Server**. Do not select the JLink entry above it: the debug probe of the board is an ST-LINK.

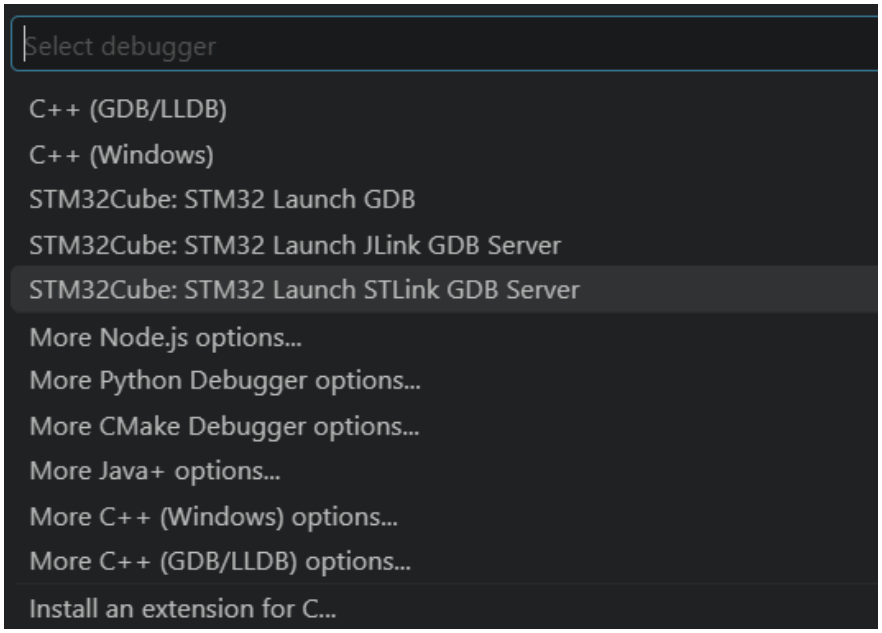


Figure 26: the Select debugger list, with STM32Cube: STM32 Launch STLink GDB Server highlighted.

In the **Run and Debug** panel, make sure that this STM32Cube launch configuration is selected. Click the green run button next to it. The STM32Cube launch type has no run-only mode: each launch opens a debug session.

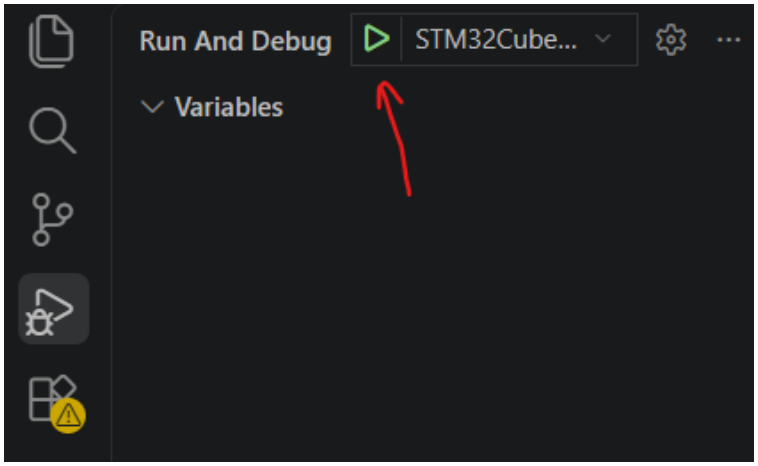


Figure 27: the run button of the Run and Debug panel, with the STM32Cube launch configuration selected.

Execution halts at the start of `main()`, at the `HAL_Init()` call. LD2 is not on yet. Press **Continue**, the second icon from the left in the debug toolbar, next to the restart icon. Execution then runs freely into the loop, and LD2 blinks.

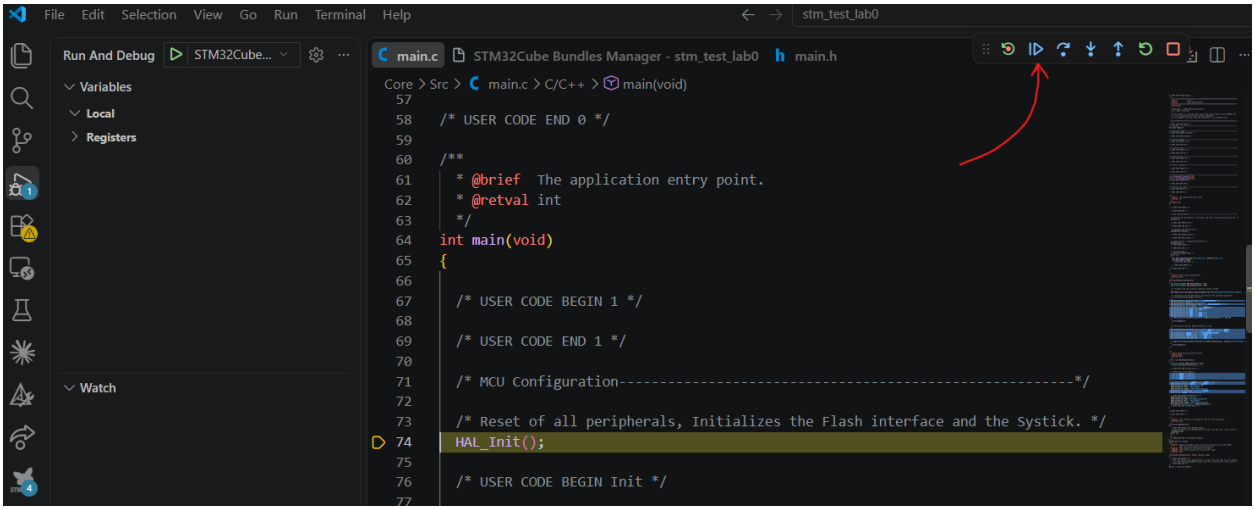


Figure 28: execution halted at the start of `main()`, with the Continue button of the debug toolbar marked.

To flash without a debug session, use the STM32CubeProgrammer command-line tool from a terminal. Its path is different on each computer. Step 7 shows how to find it. First, stop all active debug sessions, because the debugger keeps the ST-LINK connection open:

```
& "$env:LOCALAPPDATA\stm32cube\bundles\programmer\<version>\bin\
   STM32_Programmer_CLI.exe" -c port=SWD -w "build\Debug\<project-name>.
   elf" -v -rst
```

Listing 5: Flash and reset the target from PowerShell, with no debugger attached.

On macOS, run this command in a terminal, in the project folder:

```
"$HOME/Library/Application Support/stm32cube/bundles/programmer/<version>/
   bin/STM32_Programmer_CLI" -c port=SWD -w "build/Debug/<project-name>.
   elf" -v -rst
```

Listing 6: Flash and reset the target on macOS.

- `&` is the call operator of PowerShell. It is necessary to run an executable with a path in quotes.
- `-c port=SWD` connects to the target over SWD.
- `-w "..."` writes (flashes) the file into the flash memory of the target. Here, the file is the `.elf` that the Build command of Step 4 produced.
- `-v` verifies the flash contents against the file after the write.
- `-rst` resets the target when the flash operation is complete. Thus, the program starts immediately.

Step 6: Get Familiar with the Debugger

Set a breakpoint on the `HAL_GPIO_TogglePin` line: click in the margin next to its line number, or use the equivalent command of the extension. Then press Continue again. Execution now runs until it gets to that line, one loop iteration later. Thus, each new press of Continue toggles LD2 one time.

If the extension has a peripheral or register view in a debug session, open it. Look for it in the debug side panel, for example as "Peripherals" or "SFRs". Watch `GPIOA_ODR` while you continue past the breakpoint several times. Bit 5 changes between 0 and 1 at the same time as LD2. PA5 is bit 5 of the output data register of port A. This register is in the peripheral region of Figure 1.

Step 7: Verify It Worked

First, remove the breakpoint of Step 6. Press Continue. The loop then runs freely again. **Result:** LD2 blinks at 1 Hz: 500 ms on and 500 ms off.

The STM32CubeProgrammer command-line tool can show from a terminal that the board is detected. The tool is not on the system PATH. It is in the tool cache of the extension:

```
%LOCALAPPDATA%\stm32cube\bundles\programmer\<version>\bin\STM32_Programmer_CLI.exe
```

On macOS, the path is `$HOME/Library/Application Support/stm32cube/bundles/programmer/<version>/bin/STM32_Programmer_CLI`.

The extension manages four tools: ARM GCC, CMake, Ninja, and STM32CubeProgrammer. They are in this cache, each in its own subfolder with a version number. The `<version>` is different on each computer. Use this command to find the path:

```
Get-ChildItem -Path "$env:LOCALAPPDATA\stm32cube\bundles" -Recurse -Filter
    "STM32_Programmer_CLI.exe" | Select-Object FullName
```

Listing 7: Find the installed path from PowerShell.

```
find "$HOME/Library/Application Support/stm32cube/bundles" -name
   STM32_Programmer_CLI -type f
```

Listing 8: Find the installed path on macOS.

Use the `FullName` that this command gives, or the line that `find` prints, in place of the path below:

```
& "$env:LOCALAPPDATA\stm32cube\bundles\programmer\<version>\bin\
   STM32_Programmer_CLI.exe" -l
```

Listing 9: List the connected ST-LINK probes from PowerShell, with the full path to the CLI. The & at the start is the call operator of Step 5.

```
"$HOME/Library/Application Support/stm32cube/bundles/programmer/<version>/
   bin/STM32_Programmer_CLI" -l
```

Listing 10: List the connected ST-LINK probes on macOS.

A correctly connected board gives its ST-LINK serial number, firmware version, and board name under a `===== STLink Interface =====` heading:

```
-------- Connected ST-LINK Probes List --------
ST-Link Probe 0 :
   ST-LINK SN  : 003A00173433510E37363934
   ST-LINK FW  : V3J17M10
   Access Port Number  : 1
   Board Name  : NUCLEO-G491RE
-----------------------------------------------
```

Listing 11: Expected output under Connected ST-LINK Probes List. The serial number and firmware version are different on each board. The board name must be NUCLEO-G491RE.

If you see `ST-LINK error (DEV_CONNECT_ERR)` with no firmware and no board name, a debug session from Step 6 probably still locks the probe. Stop all active debug sessions in VS Code (the red square in the debug toolbar). Then try again. This closes `stlinkserver.exe` and `ST-LINK_gdbserver.exe`, which keep the connection open.

Step 8: Measure the Blink Frequency with an Oscilloscope

Connect the oscilloscope to the board while the program runs:

1. Set the attenuation switch of the probe to 10X. This decreases the load of the probe on PA5, and increases the usable bandwidth of the probe. Also set the channel input of the oscilloscope to 10X, so that the displayed voltage is correct.
2. Attach the ground lead of the probe to the `GND` pin on **CN5**. This keeps the ground return path short.
3. Put the probe tip on the `SCK/D13` pin of **CN5**, next to the `GND` pin. This pin is PA5, as the description of LD2 in UM2505 states (Figure 15).
4. On the oscilloscope, select the channel of the probe. Find the vertical scale, the timebase, and the trigger level that give a clear, stable trace.

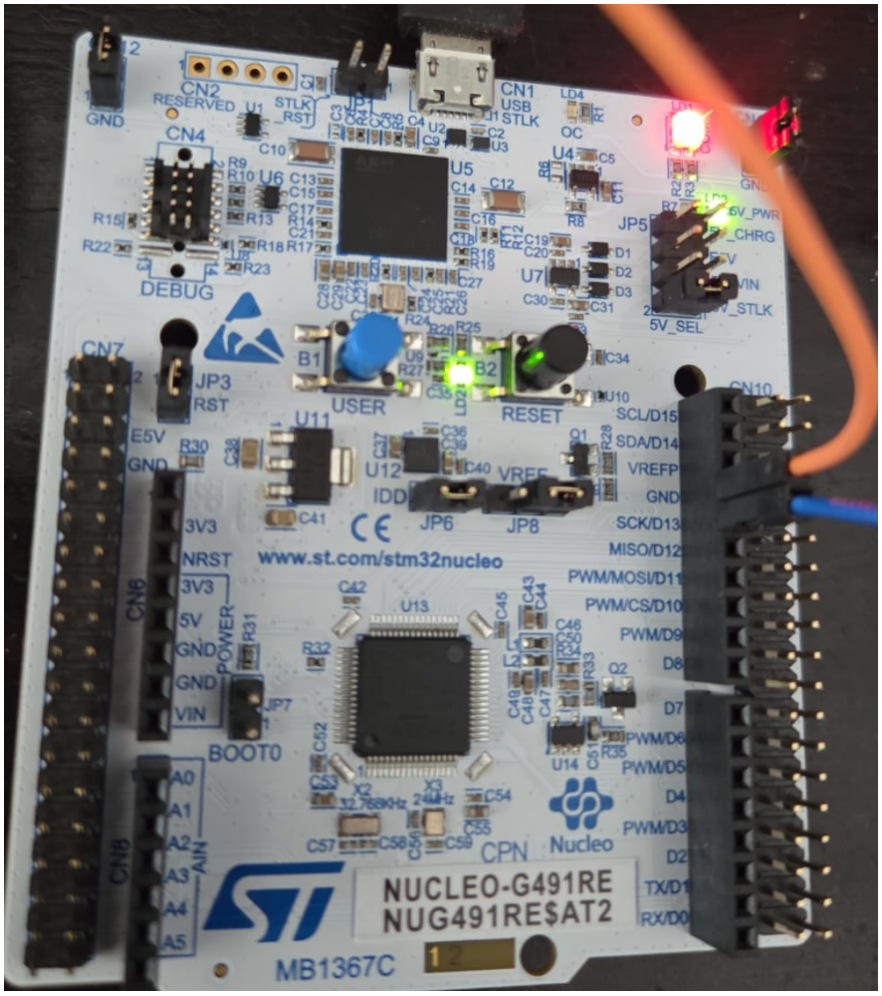


Figure 29: the probe (blue) on the `SCK/D13` pin of CN5, and the ground lead (orange) on the adjacent `GND` pin.

Make sure that LD2 blinks freely. If it does not, flash and reset the board with the terminal command at the end of Step 5. Capture the waveform. Measure the period, which is the time of one on-off cycle. Calculate the frequency, which is the reciprocal of the period.



Figure 30: a captured PA5 waveform.

Troubleshooting

The board is not detected, or "no ST-LINK probe found." In the **Run and Debug** panel, the **STM32CUBE DEVICES AND BOARDS** section shows *No device detected* (Figure 31). Make sure that the USB cable is a data cable, not a charge-only cable. Try a different USB port.

If the board is still not detected, the ST-LINK USB drivers are missing, and you must install them manually. Open the STM32Cube extension panel in the activity bar. Under **STM32CUBE RESOURCES**, select **Install ST-Link USB drivers** (Figure 32). When the installation is complete, disconnect the board and connect it again.

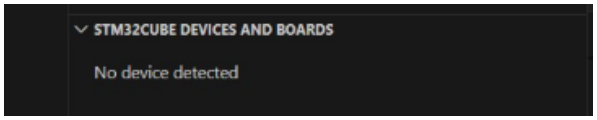


Figure 31: the STM32CUBE DEVICES AND BOARDS section of the Run and Debug panel when the board is not detected.

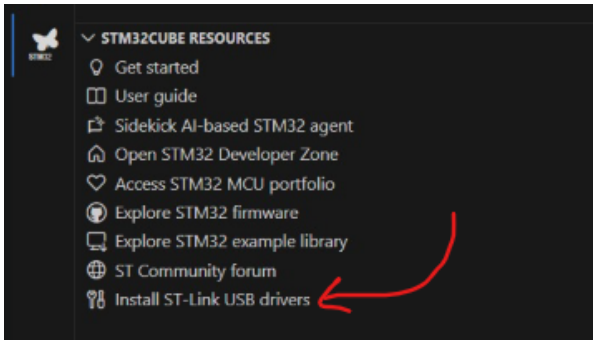


Figure 32: the installation of the ST-LINK USB drivers from the STM32Cube extension panel.

Set Up STM32Cube project(s) is gray in Step 4. Usually, the folder that is open in VS Code is not the folder

of the project. Open the folder `your-project-name/` of Figure 19, not a folder above it or inside it (**File** → **Open Folder**).

The build fails after you move or copy a project to a new folder. The CMake build cache (`build/Debug/CMakeCache.txt`) records the absolute path of the project at the time of the configuration. Delete the `build/` directory. Build the project again. CMake generates the directory again from the new location of the project.

A launch fails with a connection error. An `stlinkserver` process from a previous session can stop a new connection. End that process. Then try again. This usually solves the problem. The extension starts a new process automatically.

Debug connects, but not reliably, or an error message is about firmware. The firmware of the ST-LINK probe can be old. The extension includes a firmware upgrade tool. Run it. Then try again.

All steps are successful, but LD2 does not blink. First, make sure that you pressed Continue: each launch halts at `main()` (Step 5). Then make sure that the three edits of Step 3 are present: the two macros in `main.h`, the configuration of PA5, and the `HAL_GPIO_TogglePin` and `HAL_Delay` calls in the `USER CODE` markers.

No oscilloscope trace, or a flat line, in Step 8. Usually, the ground clip is not on a GND pin of the board. Other possible causes: the displayed channel is wrong, or the trigger level is outside the 0 to 3.3 V range of the signal. First, examine the ground connection.

STM32CubeMX does not start, or you cannot start Step 2. STM32CubeMX is a separate application (Required Hardware and Software). Install it and start it before Step 2. If it does not start, install it again.

Sign-off Checklist

- VS Code, the extension, and the board are recognized: a launch in Step 5 gets to `main()`, and `STM32_Programmer_CLI -l` shows the board in a terminal.
- The project builds with no errors.
- The student shows the first launch live: execution halts at `main()`, and one press of Continue starts the blink of LD2 at 1 Hz.
- With a breakpoint on the toggle line, the student shows two or more presses of Continue live. Before each press, the student predicts what Continue will do.
- The student captures the oscilloscope trace of Step 8 live, and gives the measured period and frequency.